

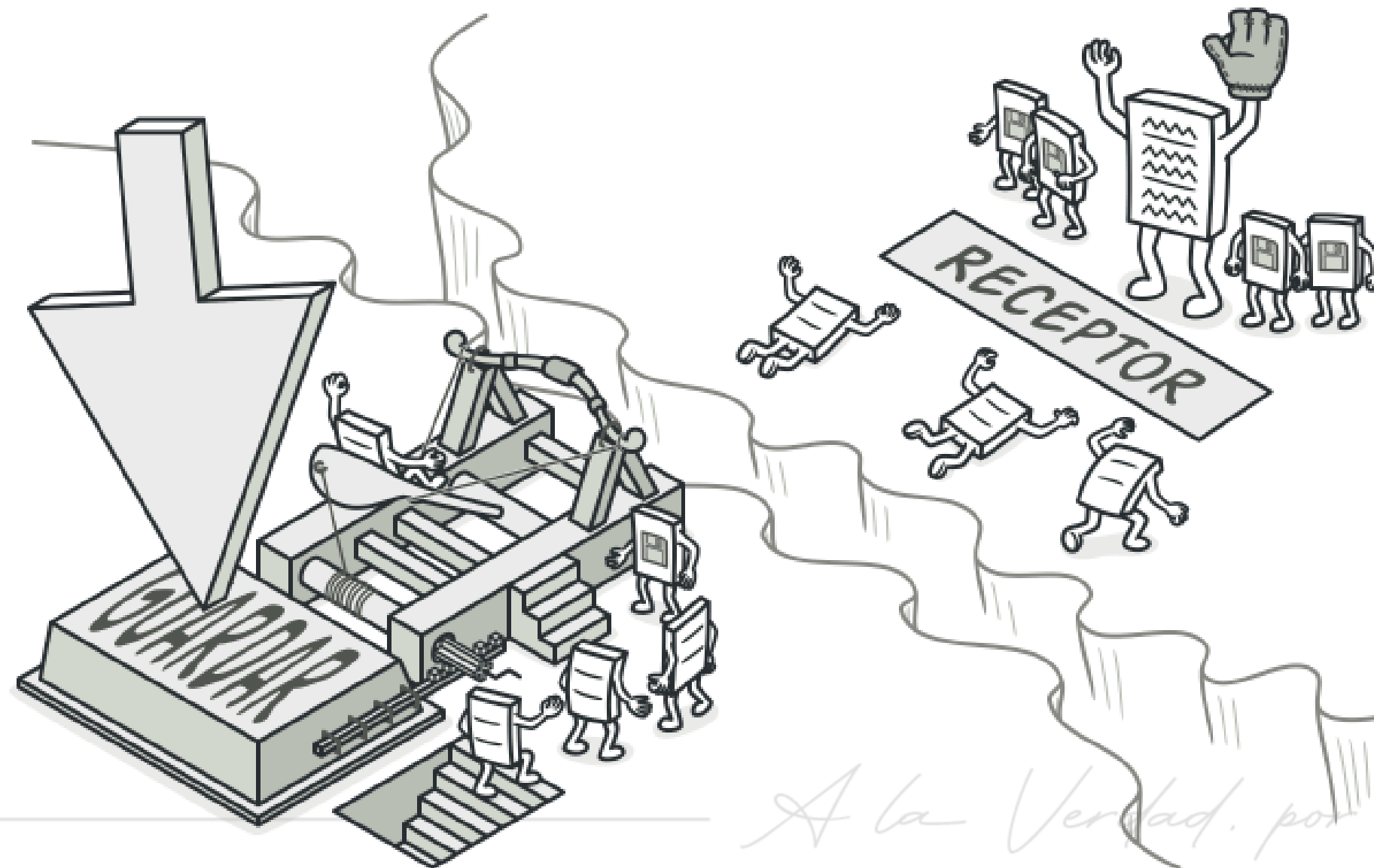


Patrones de diseño: Command

Ana Paulina Sánchez Arias

Command

Command es un patrón de diseño de comportamiento que convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.



A la Verdad, por la fe y la ciencia

Encapsula solicitudes

Convierte una solicitud en un objeto independiente, permitiendo tratarla como cualquier otro dato.

Parametriza clientes

Los clientes pueden recibir distintos comandos, encolarlos o registrarlos sin conocer su implementación.

Desacoplamiento

Separa completamente el objeto que invoca la operación del objeto que la realiza.

A la Verdad, por la fe y la ciencia

Problema

Sin el patrón Command, el código cae rápidamente en trampas difíciles de escapar:

Código acoplado

Botones directamente ligados a acciones específicas , cualquier cambio rompe todo.

Sin deshacer/rehacer

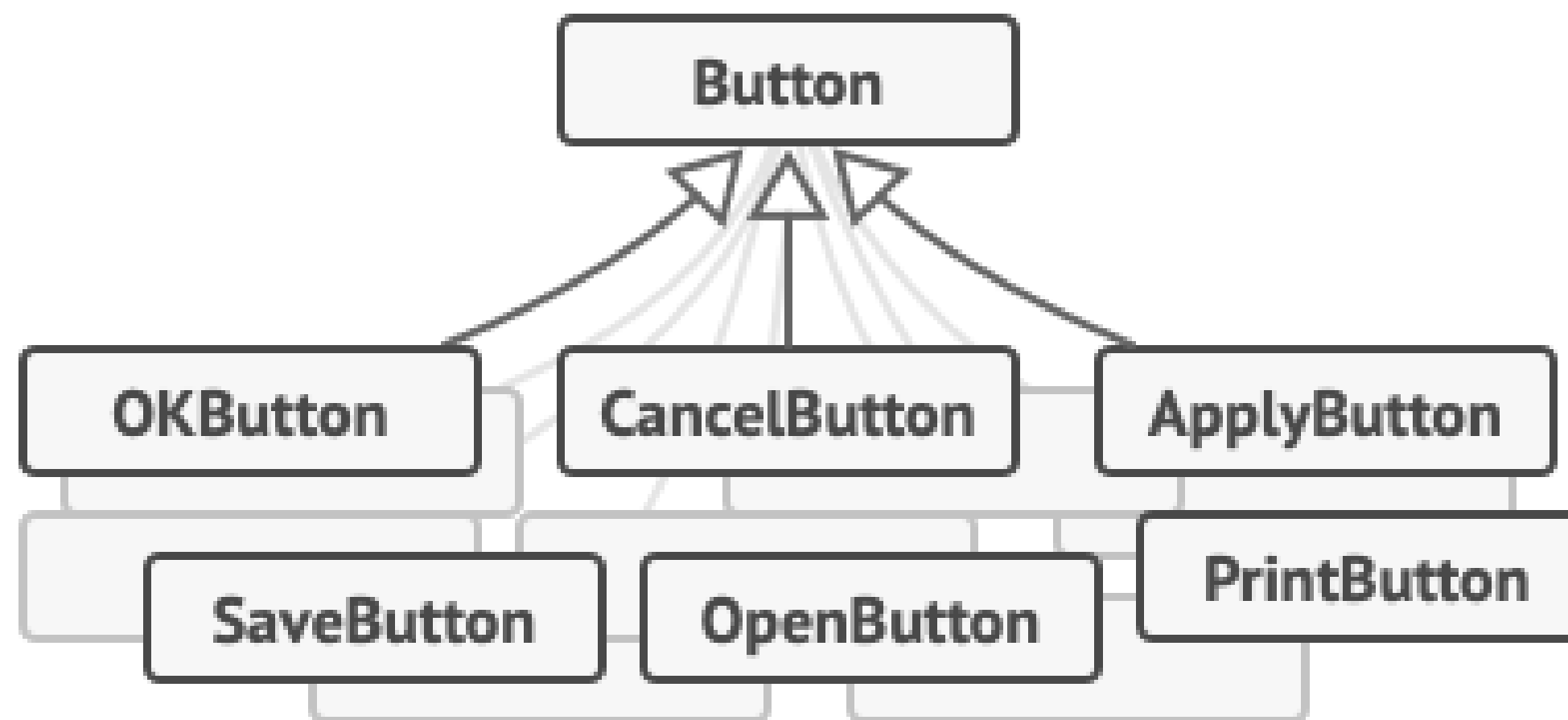
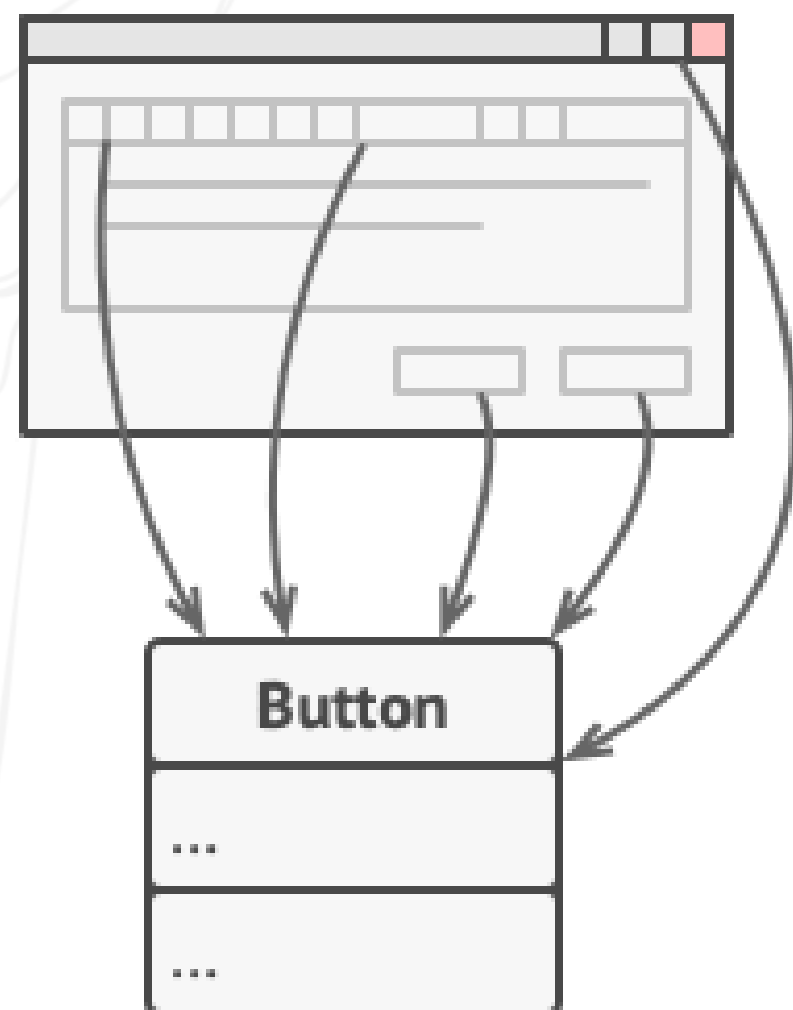
Imposible revertir operaciones sin reescribir lógica compleja.

Sin historial ni colas

No hay manera de encolar, programar o registrar acciones ejecutadas.

A la Verdad. por la fe y la ciencia

Problema



A la Verdad, por la fe y la ciencia

Componentes principales

Command

Interfaz que define la operación
Execute(). Contrato común para todos
los comandos

ConcreteCommand

Implementa Command y ejecuta la
acción real delegando al Receiver.

Invoker

Solicita la ejecución del comando sin
saber qué hace exactamente.

Receiver

El objeto que realiza el trabajo de
verdad. Contiene la lógica de
negocio.

Ventajas

- (Single Responsibility Principle): Desacopla las clases que invocan operaciones de las clases que las realizan.
- (Open/Closed Principle): Podemos introducir nuevos comandos en la aplicación sin romper el código existente de los clientes o invocadores.
- (Undo/Redo): Permite implementar mecanismos de deshacer y rehacer operaciones fácilmente.
- Permite implementar la ejecución diferida de operaciones o estructurar un sistema de colas/programación de tareas.

Desventajas

El código puede tornarse demasiado complejo.

Bibliografía

Refactoring.Guru – Patrón de diseño Command:
(<https://refactoring.guru/es/design-patterns/command>)



¡Gracias!